

QUICKSTART: Replicar Fábrica Agêntica em Novo Projeto (1 Página)

Tempo: 30 minutos | **Modelo:** Copiar → Adaptar → Validar

1 CLONAR ESTRUTURA (5 min)

```
# Option A: Copiar diretório inteiro
cp -r proj_fabrica-de-livros novo-projeto
cd novo-projeto
git remote set-url origin https://github.com/seu-usuario/novo-projeto.git
```

```
# Option B: Clonar + fazer rebase
git clone https://github.com/seu-usuario/novo-projeto.git
cp -r proj_fabrica-de-livros/.claude novo-projeto/
cp -r proj_fabrica-de-livros/scripts novo-projeto/
cp -r proj_fabrica-de-livros/templates novo-projeto/
```

2 CONFIGURAR PORTABILIDADE (5 min)

Windows:

```
.\scripts\setup-links.ps1 -Verbose
```

Mac/Linux:

```
chmod +x scripts/setup-links.sh
./scripts/setup-links.sh
```

Validar:

```
ls -la agentic/           # → .claude/
ls -la CLAUDE.md          # → hardlink
cat .mcp.json | grep mcpServers # → MCPs listadas
```

3 CUSTOMIZAR 4 ARQUIVOS (10 min)

A. Tema em CLAUDE.md

```
# Editar .claude/CLAUDE.md
# Linha 1: Trocar "Fábrica Agêntica de Publicações" → seu tema
# Linha 6: Trocar descrição
# Seção 1 (Regras): Adaptar nomes de skills se novo domínio
# Seção 2 (Squad): Trocar skills específicas de redação
```

```
# MANTER: Seção 0 (Economia Severa), RTK.md, estrutura R1–R17
```

B. Tipos de Obra

```
# Editar scripts/tipos_obra.py
# Substituir TIPOS = {...} conforme seu escopo:
```

```
# Exemplo: Para SaaS
```

```
TIPOS = {
  "feature": {
    "rotulo": "Feature / User Story",
    "raiz_output": "features",
    "natureza": "geracao",
    "custo_llm": "medio",
    "gates_conteudo": ["validar-requisitos", "validar-testes"],
  },
  "refactor": {...},
}
```

```
...  
}
```

C. MCPs

```
# Editar .mcp.json  
# Substituir paths Windows (C:\Users\...\proj_fabrica)  
# por C:\Users\seu-usuario\novo-projeto  
sed -i 's|proj_fabrica-de-livros|novo-projeto|g' .mcp.json  
  
# Validar  
cat .mcp.json | head -20
```

D. Skills (opcional)

```
# Se novo domínio requer skills específicas:  
cp .claude/skills/pesquisador .claude/skills/seu-especialista-1  
cp .claude/skills/redator-eita .claude/skills/seu-especialista-2  
  
# Editar nomes em SKILL.md (frontmatter + Skill_Nome)
```

4 VALIDAR SETUP (5 min)

```
# 1. Python syntax  
python -m py_compile scripts/*.py  
echo "✓ Python OK"  
  
# 2. CLAUDE.md carregado  
grep "alwaysApply: true" CLAUDE.md  
echo "✓ Governança OK"  
  
# 3. MCPs iniciam  
cat .mcp.json | grep -c "mcpServers"  
echo "✓ MCPs OK (deve listar N>0)"  
  
# 4. Skills callable  
ls .claude/skills/ | wc -l  
echo "✓ Skills OK (deve ter N>10)"  
  
# 5. Pre-commit hook  
test -f .git/hooks/pre-commit && echo "✓ Hook OK"  
  
# 6. Code-review-graph  
code-review-graph status 2>/dev/null && echo "✓ Grafo OK"
```

Se tudo passou: ☒ Setup completo. Prosseguir.

5 ATIVAR ECONOMIA DE TOKENS (3 min)

Copiar em .claude/ (já existem): - ☒ RTK.md (guia de economia global) - ☒ skills/caveman (brainstorm: 60% economia) - ☒ skills/headroom (logs: 75% economia) - ☒ skills/lean-ctx (reads: 40% economia) - ☒ skills/rtk-memory (rastreamento) - ☒ skills/pre-flight-check (evita loops)

Testar:

```
/caveman "qual é a melhor abordagem?"  
# → deve responder em 3–5 linhas telegráficas
```

```
/headroom
```

```
# → mostrar skill para comprimir logs

/lean-ctx
# → mostrar skill para grep + read otimizado
```

6 PRIMEIRO COMMIT (2 min)

```
git add -A
git commit -m "chore(init): fábrica replicada + customização de escopo"

git log --oneline -1
# → visualizar commit

git push origin main
# → sincronizar
```

7 PRÓXIMOS PASSOS

Fase	O Que Fazer	Tempo
Phase 1 (P&D)	Invocar pesquisador ou skill equivalente	30 min
Phase 2 (Manufatura)	pool-capitulos.py --plano + subagentes paralelos	2-4h
Phase 2.5 (Review)	auditar-obra.py --estrito + correções	1h
Phase 3 (Entrega)	Compilador + PDF	30 min
Phase 4 (Opcional)	Coleção + derivadas (playbook, deck, etc.)	2h

 CHECKLIST RÁPIDO

- CLONE & PORTABILIDADE
- [] Diretório clonado/copiado
 - [] setup-links.{ps1,sh} executado
 - [] agentic/ → .claude/ (junction OK)
 - [] CLAUDE.md (hardlink OK)
- CUSTOMIZAÇÃO
- [] CLAUDE.md: tema, squad, skills específicas
 - [] scripts/tipos_obra.py: tipos de artefato
 - [] .mcp.json: paths ajustados
 - [] Skills especializadas criadas (se aplicável)
- VALIDAÇÃO
- [] Python syntax OK
 - [] CLAUDE.md carregado (alwaysApply)
 - [] MCPs reconhecidas
 - [] Skills listadas
 - [] Pre-commit hook presente
 - [] Code-review-graph status OK
- TOKEN ECONOMY
- [] RTK.md copiado
 - [] Skills de economia (caveman, headroom, lean-ctx) presentes
 - [] Testadas manualmente
- ENTREGA
- [] Commit inicial feito

- [] Push para remote OK
- [] Pronto para Phase 1 (P&D)

 TROUBLESHOOTING RÁPIDO

Problema	Solução
agentic/ não funciona	Rodar setup-links.ps1 novamente (admin)
.mcp.json com paths incorretos	sed -i 's proj_fabrica seu-projeto g' .mcp.json
code-review-graph status falha	Instalar: uvx code-review-graph serve
Python syntax error em scripts/	python -m py_compile scripts/*.py (verificar qual arquivo)
CLAUDE.md não carrega	Verificar YAML frontmatter: --- no topo + alwaysApply: true
Skills não aparecem em /help	Verificar .claude/skills/*/SKILL.md (frontmatter correto)

 ANTES & DEPOIS

Métrica	Sem Replicação	Com Replicação
Setup	4–6 semanas (do zero)	30 minutos
Skills	0	50+ (prontas)
Comandos	0	19 orquestradores
MCPs	0	4 + extensível
Token Economy	Nenhuma	50–70% economia
Templates	Nenhum	Markdown, Typst, HTML
Portabilidade	Nenhuma	Multi-IDE (5 IDEs)
Gates de Qualidade	0	8+ gates determinísticas

 RESULTADO FINAL

Após 30 minutos: -  Projeto estruturado como Fábrica -  Governança central (CLAUDE.md) -  50+ skills prontas -  Economia de tokens ativa (caveman, headroom, lean-ctx) -  Portabilidade multi-IDE -  Pronto para Phase 1

Próximo: /criar-livro "<seu-tema>" (ou comando equivalente do seu escopo)

Dúvidas? Consultar: - 2026-08-23-plano-replicacao-componentes-fabrica.md (visão completa) - 2026-08-23-economia-tokens-blueprint.md (detalhes de economia) - .claude/CLAUDE.md (governa Este projeto)